2023 3rd International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI) | 979-8-3503-7205-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/CEI60616.2023.10527829

Convergence Analysis of Different Categories of Gradient Descent Algorithms

Ruitong Han
Faculty of Engineering university of Bristol, Bristol, UK
1581136712@qq.com

***Abstract*—With the rapid development of neural network models, gradient descent as the most important optimization algorithm in neural network models has received extensive attention from academia. At present, neural network technology has been used in various fields, such as computer vision, natural language processing, and speech recognition. In recent years, many kinds of gradient descent algorithms have been proposed to improve existing algorithms, each with higher efficiency and a more stable model structure. This paper will summarize and analyze various gradient descent algorithms and focus on the convergence of various algorithms to find a better method and analyze the optimization ability of that algorithm.**

Keywords*—Deeping Learning, Gradient Descent, Deep Neural Network, Machine Learning, Convergence.

I. INTRODUCTION

In modern daily life, the word 'smart' is becoming easier to find, such as smartphones, smart watches, smart driving, and so on. At the same time, the emergence of the term artificial intelligence also means that machines will acquire human intelligence shortly, and artificial intelligence has become a branch of the computer field [1]. The main part of artificial intelligence is machine learning, and only when the machine learns the ability of human beings, the machine can become more capable of meeting the needs of people's daily life. Machine learning means how to further generalize and apply the scientific laws derived from the summary to other unobserved learning samples [1].

After entering the 21st century, with the development of science and technology, computer hardware performance and computing power are constantly increasing [2], which to a certain extent promotes the development of neural networks, deep neural network-related research has slowly begun to appear, and around 2010 began the explosion of deep neural network development, deep neural network applications are becoming more and more extensive, the following will list some applications of deep neural networks, from these applications can also be seen that deep neural networks will bring great impact to people's lives, will make people's lives more and more convenient;

Video and images: The use of deep neural network technology can effectively improve the accuracy of many computer vision tasks, such as image classification, target localization, and monitoring, image segmentation, and pose recognition [2].

Speech and natural language: Deep neural network technology has been used for speech recognition and has significantly improved the accuracy of recognition; In addition, it is also used to handle many speech and language-related tasks, such as machine translation, natural language processing, and audio generation [3].

Medicine: Deep neural networks also play an important role in genetics, researchers use them to understand some diseases, such as autism, cancer, and spinal muscular atrophy. Moreover, deep neural networks can also be used in medical images, such as to detect skin cancer, brain cancer, and breast cancer [2].

Robotics: Deep neural network technology has also achieved great success in the field of machine control, and researchers have used deep neural network technology to solve the problem of gripping robotic arms, the motion planning problem of ground robots, the problem of visual navigation, the stability control problem of quadcopters, and the driving strategy of self-driving cars [4].

From the MP model in 1943 to today's deep neural network, the artificial neural network has experienced more than half a century of development. The current application field of neural networks is becoming more and more extensive, people try to use neural network technology to solve problems, and gradient descent is an important algorithm in machine learning. Among them, gradient descent will be divided into different types of gradient descent methods according to the loss function and learning rate, such as batch gradient descent; stochastic gradient descent; momentum gradient descent. This article will focus on the convergence of different gradient descents and analyze their differences to find one or more relatively excellent gradient descent algorithms.

II. METHOD

Machine learning is an interdisciplinary discipline that integrates multiple fields and is the core field of artificial intelligence. Machine learning focuses on how computers can simulate human learning behavior, gain experience from historical data, and apply it to new scenarios [5].

Machine learning learns from the training dataset, and according to the characteristics of the training data, machine learning can be roughly divided into supervised learning, semi-supervised learning, unsupervised learning, and weakly supervised learning [5]. In the case of supervised learning, for example, each training data has a label. After the machine learning model is determined based on the scenario of the application, you can input samples from the training dataset into the machine learning model, and the machine learning model outputs the corresponding prediction labels [6]. To measure the correctness of the prediction results, the loss function needs to be used to represent the gap between the predicted value and the true label. With the loss function, the problem to be solved

Authorized licensed use limited to: BMS Institute of Technology. Downloaded on May 05,2026 at 06:15:48 UTC from IEEE Xplore. Restrictions apply.

by machine learning can be expressed as optimizing an objective function:

$$\min_{w \in R^d} F(w) = \frac{1}{N} \sum_{i=1}^{N} f(w; s_i) \quad (1)$$

where N is the number of data samples in the entire training dataset, s_i represents the ith data sample in the dataset, w represents the model parameters, $f(w; s_i)$ represents the loss function on the model parameters w and data samples s_i. $F(w)$ represents the loss function on the entire dataset, which is also called empirical risk. s_i can generally be expressed as (x, y), where represents the input vector x_i then represents its corresponding label y_i which is a scalar. The loss function $f(w; s_i)$ measures the difference between the predicted value predicted by the machine learning model and the true label when the input value is x_i.

When the dataset is large or the model complexity is high, it can take a long time to perform calculations at a single node, or a single node simply cannot store the entire dataset or machine learning model. At this time, it is necessary to use multiple nodes to learn.

Assuming that the entire data set is randomly divided into all worker nodes, according to Equation (1), the problem to be solved by distributed machine learning can be further expressed as

$$\min_{w \in \mathbb{R}^d} F(x) = \frac{1}{P} \sum_{i=1}^{P} F_i(w) \quad (2)$$

where $F_i(w)$ represents the loss function on the worker node i.$F_i(w)$ can be expressed as:

$$F_i(w) = \frac{1}{|D_i|} \sum_{s \in D_i} f(w; s_i) \quad (3)$$

Where D_i represents the local dataset on the worker node i and $|D_i|$ represents the sample size of that local dataset.

To put a single node machine learning model into a distributed environment for training, you need to split the dataset used for training or the trained machine learning model into multiple nodes. Depending on whether the model or data is split, the implementation of distributed machine learning can be divided into model parallelism and data parallelism. When the scale of the machine learning model is large and cannot be stored on a single worker node, the model needs to be divided into multiple worker nodes, that is, the models are parallel, and the schematic diagram is shown in Figure 1. In model parallelism, because the entire model is split into individual worker nodes, the entire training process cannot proceed when a worker node goes down [7].

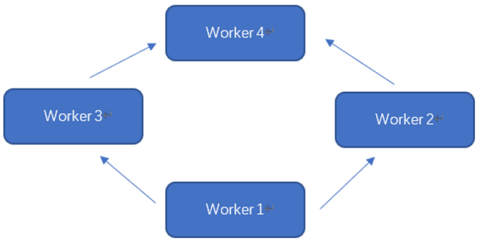


Figure 1. Model parallel diagram

When the entire training dataset is large and cannot be stored in a single worker node, the training dataset needs to be divided into multiple worker nodes, that is, the data is parallel, and the schematic diagram is shown in Figure 2. In data parallelism, all worker nodes maintain the entire learning model locally, calculate local model update parameters based on local datasets, and cooperatively maintain a global parameter through communication [7].

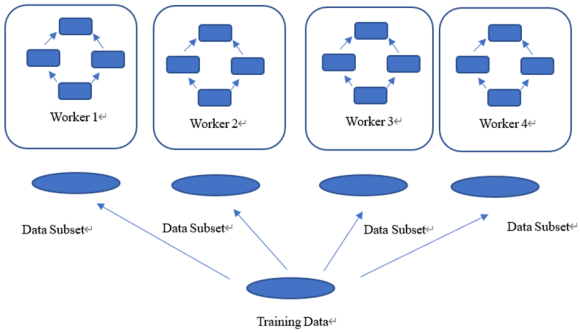


Figure 2. Data parallelism diagram

The Gradient Descent (GD) algorithm updates the parameters once for the entire dataset, which is too computational, the training time is slow and it is easy to fall into the local optimal value.

Stochastic Gradient Descent (SGD) is to randomly draw training samples from the entire data set participating in training, and the calculated loss is from randomly sampled samples to complete the gradient update. Due to frequent updates so that the parameters have high variance, the loss function fluctuates with different intensities and overshoots due to frequent fluctuations [8].

In view of the drawbacks of gradient descent algorithms, the Mini Batch Gradient Descent (MBGD) algorithm has emerged. The batch gradient descent algorithm divides the entire training dataset into several small datasets of similar size, trains the model with one small dataset at a time, and calculates the loss to update the gradient of the model. Next, use the next batch of small datasets to accomplish the same operation until all batches of datasets are exhausted. When small datasets are well partitioned enough, the time cost of calculating the loss function and the complexity of model training are greatly reduced. In general, the minibatch gradient descent algorithm is also called the stochastic gradient descent algorithm. The stochastic gradient descent method update formula is as follows:

$$w_{(t+1)} = w_t - \eta \nabla f(w; s) \quad (4)$$

Because stochastic gradient descent calculates stochastic gradients by taking only one sample at a time, the time spent on each update is greatly reduced. However, the stochastic gradient of a sample does not represent the gradient on the entire dataset, so the stochastic gradient descent method does not carry out every iteration in the negative direction of the full gradient, and the convergence process is relatively jittered [9].

Since the stochastic gradient of a single sample and the full gradient of all samples differ greatly, the number of iterations required for convergence increases greatly when using

Authorized licensed use limited to: BMS Institute of Technology. Downloaded on May 05,2026 at 06:15:48 UTC from IEEE Xplore. Restrictions apply.

stochastic gradient descent. Therefore, the researchers proposed the small-batch stochastic gradient descent method, that is, the gradient of a part of the sample is selected each time to update the model parameters, and the update formula is as follows:

$$w_{(t+1)} = w_t - \eta \nabla f(w_t; \xi_t) \tag{5}$$

where ξ represents a random sample of t a batch selected at the time of the first iteration, assuming the batch size is m, yielding:

$$f(w_t; \xi_t) = \frac{1}{m} \sum_{s \in \xi_t} f(w_t; s) \tag{6}$$

In the original backpropagation algorithm, the gradient descent method is used for weight update, but the gradient descent method is good but also has its disadvantages, because each update of the gradient descent method is based on the current position, so the gradient descent method to the most advantageous update path will be more tortuous, so that the speed of update convergence is very slow, often need many iterations of the process network can converge; The momentum gradient descent law is an improvement on it, its main idea is to update the weight of the network to find the optimal solution process as a ball from the mountain down the process of falling, we know that due to gravity, the ball will roll down more and more, the speed of the ball will change the position of the ball, the loss function on the weight parameter gradient as gravity, through exponential weighted average to accumulate the previous gradient and then update the parameters, the momentum gradient descent method update formula is as follows:

$$v_{\Delta w}^{k+1} = \beta v_{\Delta w}^{k} + (1-\beta)\Delta w^{k}$$
$$v_{\Delta b}^{k+1} = \beta v_{\Delta b}^{k} + (1-\beta)\Delta b^{k}$$
$$w^{k+1} = w^{k} - a v_{\Delta w}^{k+1}, b^{k+1} = b^{k} - a v_{\Delta b}^{k+1} \tag{7}$$

where a is the learning rate, β is the momentum coefficient, is a hyperparameter that can be set by itself, usually set to 0.9, can also be set according to the situation when using, doctors find that when $\beta = 0$, the updated formula of the momentum gradient descent method is consistent with the gradient descent method, and when $\beta = 0.9$, expand the formula to get:

$$w^{k+1} = w^{k} - a v_{\Delta w}^{k+1}$$
$$= w^{k} - a[\beta v_{\Delta w}^{k} + (1-\beta)\Delta w^{k}]$$
$$= w^{k} - a\{\beta[\beta v_{\Delta w}^{k-1} + (1-\beta)\Delta w^{k-1}] + (1-\beta)\Delta w^{k}\}$$
$$= w^{k} - [a\beta^{k+1} v_{\Delta w}^{0} + a(1-\beta)\sum_{i=0}^{k} \beta^{i} \Delta w^{k-i}]$$
$$= w^{k} - a\sum_{i=0}^{k} 0.1 \times 0.9^{i} \Delta w^{k-i}$$

From the formula, we can see that the momentum gradient descent method actually makes an exponential weighted average of all gradients in history, and the gradient farther from the current number of iterations gives a smaller weight coefficient, while the further forward the weight coefficient is almost negligible. This update method means that it will not immediately change the direction of gradient optimization, each optimization direction is the exponential weighted average of the historical direction, it is changed and updated by accumulating little by little, so that each time the gradient is obtained through different training examples, the gradient value in the optimal direction will always be increased, so that the network will not produce a lot of oscillations when iteratively updated, so as to achieve the effect of accelerated convergence.

III. Convergence of Gradient Descent

The computing and storage resources of a single machine are limited, so it is necessary to use multiple machines for learning, and the stochastic gradient descent method is an optimization algorithm on a single machine, in order to apply the stochastic gradient descent method to a distributed environment, it is necessary to design the corresponding distributed stochastic gradient descent method. The following describes several typical distributed stochastic gradient descent methods under the parameter server architecture.

Each worker node needs to communicate with the parameter server, so that the final model trained by the parameter server is the optimal model based on the entire dataset. When the communication between the worker nodes and the parameter server is synchronized, such a method is called synchronous stochastic gradient descent (SSGD) [10].

In SSGD, the parameter server first issues an initial model to all worker nodes, and all worker nodes will calculate a stochastic gradient based on the model and the local dataset after receiving the initial model. The calculation of the stochastic gradient is generally carried out by selecting one or a batch of data samples, assuming that each worker node extracts m data samples when calculating the stochastic gradient, then the formula for calculating the local stochastic gradient for each worker node is.

$$g(w_t, \xi_t^i) = \frac{1}{m} \sum_{s \in \xi_t^i} \nabla f(w_t, s) \tag{8}$$

where ξ_t^i represents the set of data samples with a batch size m of the ith worker sampled by the tth iteration, $g(w_t, \xi_t^i)$ representing the stochastic gradient calculated on this data sample set.

After completing the calculation task of the stochastic gradient, each worker node uploads the calculation result of the gradient to the parameter server. After the parameter server collects all the minion's calculation results, it will iteratively update the global model on the server side, and the update rules are as follows:

$$w_{t+1} = w_t - \eta \frac{1}{p} \sum_{t=1}^{p} g(w_t; \xi_t^i) \tag{9}$$

where p represents the number of worker nodes, ξ_t^i represents the data sample set with batch size m from the ith worker node sampling at the tth iteration, $g(w_t; \xi_t^i)$ represents the average gradient calculated on this data sample set, and η represents the learning rate. After the parameter server updates the model, it delivers the new model to all worker nodes. After the minion gets the new model from the parameter server, it can continue with local iterative updates.

Authorized licensed use limited to: BMS Institute of Technology. Downloaded on May 05,2026 at 06:15:48 UTC from IEEE Xplore. Restrictions apply.

ALGORITHM 1: Synchronous stochastic gradient descent

Minion nodes $i = 1,2,\ldots,P$;
repeat
Get the model from the parameter server
A random m training sample is drawn from the local dataset and the stochastic gradient is calculated according to the formula (8).
Upload the stochastic gradient to the parameter server $g(w,\xi)$
Until convergence
Parameter server:
input: Learning rate η
initialize: when $t = 0$, initialize the model w_0
repeat
When a stochastic gradient is received from all worker nodes, the model is updated according to the formula (9).
$t \leftarrow t + 1$
Deliver the model w_t to all worker nodes.
Until convergence

After completing the local calculation task, the worker node uploads the calculation result to the parameter server, and the parameter server updates the model parameters after receiving the random gradient of any node. The update rules for the parameter server node model are:

$$w_{t+1} = w_t - \eta g(w_{t-\tau_t}; \xi_t^{i(t)}) \quad (10)$$

where $i(t)$ represents the index of the minions t at the time of the model update, which provides the stochastic gradient required for the model update. $\xi_t^{i(t)}$ represents the data sample set of the batch size sampled by the $i(t)$ worker node during the tth iteration, $g(w_{t-\tau_t}; \xi_t^{i(t)}) \approx \frac{1}{m}\sum s \in \xi_t^{i(t)} \nabla f(w_{t-\tau_t}; \xi_t^{i(t)})$ representing the stochastic gradient on the data sample set, η representing the learning rate, and τ_t representing the delay of the tth iteration.

In order to be able to perform convergence analysis, this article first makes the following commonly used assumptions.

Hypothesis 1: suppose $F(w)$ is smooth on $\mu -$. Therefore, for arbitrary w, w'.

$$F(w) - F(w)' \le \langle \nabla F(w), w' - w \rangle + \frac{\mu}{2}\|w' - w\|^2 \quad (11)$$

Hypothesis 2: suppose $F(w)$ is strongly convex on $c -$. Therefore, for any of the w, w'

$$F(w) - F(w)' \ge \langle \nabla F(w), w' - w \rangle + \frac{c}{2}\|w' - w\|^2 \quad (12)$$

Because $F(w)$ is strongly convex, $F(w)$ has a unique minimum value, denoting by w_* at point, then for any w,

$$2c(F(w) - F(w_*)) \le \|\nabla F(w)\|^2 \quad (13)$$

Hypothesis 3: The data set randomly divided into all worker nodes is independently and homogeneous, and the random gradient of the sample on any node is an unbiased estimate of the true gradient. That is, for any w

$$\mathbb{E}\ s[\nabla f(w; s)] = \nabla F(w) \quad (14)$$

where s represents any randomly sampled data sample.

Hypothesis 4: The variance of a stochastic gradient is bounded. That is, for any w,

$$\mathbb{E}\ s\|\nabla f(w; s) - \nabla F(w)\|^2 \le \sigma^2 \quad (15)$$

Lemma 3.1 If $\mathbb{E}\ s\nabla f(w; s) = \nabla F(w)$ and $\mathbb{E}\ s\|\nabla f(w; s) - \nabla F(w)\|^2 \le \sigma^2$, can be obtained

$$\mathbb{E}\ \xi_t^i \left\| g(w_{t-\tau_t}; \xi_t^i) - \nabla F(w_{t-\tau}) \right\|^2 \le \frac{\sigma^2}{m} \quad (16)$$

Where $\mathbb{E}\ \xi_t^i$ represents the expectation for the sample dataset ξ_t^i of the t worker node of the i iteration.

Lemma 3.2 if $\mathbb{E}\ s\nabla f(w; s) = \nabla F(w)$ and $\mathbb{E}\ s\|\nabla f(w; s) - \nabla F(w)\|^2 \le \sigma^2$, can be obtained

$$\mathbb{E}\ \xi_{t,*} \left\| \frac{1}{|\varsigma_{k(t)}|}\sum_{i\in\varsigma_{k(t)}} g(w_{t-\tau_t}; \xi_t^i) \right\|^2 \le \frac{\sigma^2}{m|\varsigma_{k(t)}|} + \left\| \nabla F(w_{t-\tau_t}) \right\|^2 \quad (17)$$

Lemma 3.3 If the learning rate is satisfied as $\eta < max(\frac{1}{\mu}, \frac{1}{c})$, then [11].

$$\mathbb{E}\ [F(w_t) - F(w_*)] \le a'\mathbb{E}\ [F(w_0) - F(w_*)] + \frac{\mu\sigma^2\eta^2}{2m}\sum_{i=0}^{t-1}\frac{a^i}{\left||\varsigma_{k(t-1-i)}|\right|} \quad (18)$$

Thereinto $a = 1 - \eta c(1 - \gamma)$. Since, $\eta \le \frac{1}{\mu}$, $0 \le \gamma \le \frac{1}{c}$, $0 \le a < 1$the inequality (18) is expected to be obtained for all randomly sampled samples in the fth iteration

$$\mathbb{E}\ \xi_{t,*}[F(w_{t+1}) - F(w_*)] \le \mathbb{E}\ \xi_{t,*}[F(w_t) - F(w_*)] + \mathbb{E}\ \xi_{t,*}\frac{\mu}{2}\left\| \eta\frac{1}{|\varsigma_{k(t)}|}\sum_{i\in\varsigma_{k(t)}} g(w_{t-\tau_t}; \xi_t^i) \right\|^2 + \mathbb{E}\ \xi_{t,*} < \nabla F(w_t), -\eta\frac{1}{|\varsigma_{k(t)}|}\sum_{i\in|\varsigma_{k(t)}|} g(w_{t-\tau}; \xi_t^i) > \quad (19)$$

For this term $\mathbb{E}\ \xi_{t,*}\frac{\mu}{2}\left\| \eta\frac{1}{|\varsigma_{k(t)}|}\sum_{i\in\varsigma_{k(t)}} g(w_{t-\tau_t}; \xi_t^i) \right\|^2$ can be obtained from lemma 3.2

$$\mathbb{E}\xi_{t,*}\frac{\mu}{2}\left\| \eta\frac{1}{|\varsigma_{k(t)}|}\sum_{i\in|\varsigma_{k(t)}|} g(w_{t-\tau_t}; \xi_t^i) \right\|^2 \le \frac{\mu\sigma^2\eta^2}{2m|\varsigma_{k(t)}|} + \frac{\mu\eta^2}{2}\left\| \nabla F(w_{t-\tau_t}) \right\|^2 \quad (20)$$

One of them

$$\mathbb{E}\ \xi_{t,*} < \nabla F(w_t), -\eta\frac{1}{|\varsigma_{k(t)}|}\sum_{i\in|\varsigma_{k(t)}|} g(w_{t-\tau}; \xi_t^i) >$$

$$=< \nabla F(w_t), -\eta \sum_{i\in\varsigma_{k(t)}} \frac{1}{|\varsigma_{k(t)}|}\frac{1}{m}\sum_{s_j\in\xi_{i,j}} E_{s_j}\nabla f(w_{t-\tau}; s_j) >$$

$$=< \nabla F(w_t), -\eta\nabla F(w_{t-\tau_t}) > \quad (21)$$

Bring inequalities in

$$\mathbb{E}\ \xi_{t,*}[F(w_{t+1}) - F(w_*)] \le \mathbb{E}\,\xi_{t,*}[F(w_t) - F(w_*)] - \eta < \nabla F(w_t), \nabla F(w_{t-\tau}) >$$

Authorized licensed use limited to: BMS Institute of Technology. Downloaded on May 05,2026 at 06:15:48 UTC from IEEE Xplore. Restrictions apply.

$$+\frac{\mu\sigma^2\eta^2}{2m|\varsigma_{k(t)}|}+\frac{\mu\eta^2}{2}\left\|\nabla F(w_{t-\tau_t})\right\|^2$$

Since $< a,b >= \frac{1}{2}(\|a\|^2+\|b\|^2-\|1-b\|^2)$, the expected value is evaluated on both sides at the same time and the final result is

$$\mathbb{E}\ [F(w_{t+1})-F(w_*)]$$

$$\overset{(a)}{\leq}\mathbb{E}\,[F(w_t)-F(w_*)]-\frac{\eta}{2}(1-\gamma)\ \mathbb{E}\ \|\nabla F(w_t)\|^2+\frac{\mu\sigma^2\eta^2}{2m|\varsigma_{k(t)}|} \quad (21)$$

$$\overset{(b)}{\leq}(1-\eta c(1-\gamma))\mathbb{E}[F(w_t)-F(w_*)]+\frac{\mu\sigma^2\eta^2}{2m|\varsigma_{k(t)}|} \quad (22)$$

So it can be obtained from theorem 3.3 that in GSGD, the error decays t at a rate η relative to the number of a^t iterations, and when a fixed learning rate is used, the error can eventually converge to a fixed upper bound

$$\frac{\mu\sigma^2\eta^2}{2m|\varsigma_{k(t)}|}\lim_{t\to+\infty}\sum_{t=0}^{t-1}\frac{a^t}{|\varsigma_{k(t-1-i)}|}$$

The above theorem is general and applies equally to SSGD and ASGD. For SSGD, the attenuation rate $(1-\eta c)^t$ of the error is $\gamma=0$, and the upper bound to which the final error converges is $\frac{\mu\sigma^2\eta}{2mcP}$, since all worker nodes are in the same group. For ASGD, γ is a large value, and since each worker node can be treated as a group $\forall t, |\varsigma_{k(t)}|=P$, the upper bound of the final error convergence is $\frac{\mu\sigma^2\eta}{2mcP(1-\gamma)}$.

IV. Conclusion

Distributed machine learning is of great significance for the development of machine learning, and stochastic gradient descent is one of the most commonly used optimization algorithms in machine learning. When the stochastic gradient descent method is used in a distributed environment, the convergence speed of the algorithm on time is slow due to the difference in the performance of each worker node or the additional overhead introduced by the communication of the worker nodes. In this topic, how to improve the convergence performance of the stochastic gradient descent method in distributed machine learning, so that the algorithm can converge quickly while ensuring the accuracy of the final model

References

[1] Debing Wu. Convergence Analysis of Gradient Clipping Descent Algorithm. 2022

[2] Yuyu Zhu. Optimization Algorithm of Convolutional Neural Network Based on Fractional Calculus. 2022

[3] Llechun y, Jackel L D, Boset B, et, al. Handwrit digit recognition: application of neural network chips and automatic learning[J]. IEEE Commun Mag. 1989. 27(11):41-46

[4] Kirzhevsky A, Sutskever I, Hinton GE, Imagenet Classification with deep Convolutional Neural Network [A]? NIPS, 2012

[5] J. Long, E. Shenlhamer, and T. Darrell, 6CFully Convolutional Networks for semantic Segmentation. [A], CVPR, 2015

[6] C. Chen A. Seff, A. Komhauser, and J. Xiao. 6tDeepdriving: Learning affordance for direct perception in autonomous driving. [A]. ICCV. 2015

[7] T. Zhang. G. Kahn. S. Levine, and P. Abbeel. Learning Deep Control Policies for Autonomous Aerial Vehicles with MPC-guided Policy search9, [A]. ICRA, 2016

[8] Rumelhert D E, McClelland J L, eds, Parallel Distributed Processing: Explorations in the Microstructure of Cognition. Vol. 1. Cambrige. MA: MIT Press, 1986

[9] Crema A, Loreto M, Raydan M, Spectral Projected Subgradient with a Momentum term for the Lagrangean Dual Approach [J]. Computers and Operations Research, 2007 34 (10) 3174-3186

[10] Zhang W. GUPTA S. Lian X5 et al, Staleness-aware async-sgd for distributed deep learning [C]//Proceedings of the Twenty-Fifth Inter national Joint Conference on Artificial Intelligence. 2016: 2350-2356

[11] HANG H. LI J3 KARA K5 et al.Zipml: Training Linear models with end-to-end low precision and a little bit of deep learning [C]//Intemational Conference on Machine Learning. PMLR, 2017: 4035-4043

Authorized licensed use limited to: BMS Institute of Technology. Downloaded on May 05,2026 at 06:15:48 UTC from IEEE Xplore. Restrictions apply.